

Aplicação de Conversação

FASE 2 – PROJETO DE LABORATÓRIOS DE TELECOMUNICAÇÕES E INFORMÁTICA I

Mestrado Integrado em Engenharia de Telecomunicações e Informática

Grupo 3:

- Leandro Henrique Dantas Alves A82157
- André Martins Almeida A82211
- José Eduardo da Silva Santos A82350



Índice

Índice	2
Índice de figuras e tabelas.....	3
1 Introdução	4
1 Tarefa 2.1 – Comunicação por radiofrequência Arduino-TRF-TRF-Arduino..	5
1.1 Interface de comunicação série TRF-Arduino.....	5
1.2 Ligações físicas	7
1.3 Impacto dos diferentes parâmetros configuráveis de TRF	7
1.4 Desenvolvimento do código nos módulos Arduino	8
1.5 Testes experimentais	9
2 Tarefa 2.2 – Controlo de acesso ao meio.....	11
2.1 Situações em que se torna necessário utilizar um protocolo de acesso ao meio (MAC)	11
2.2 Protocolos MAC para redes sem fios.....	11
2.2.1 ALOHA.....	11
2.2.2 CSMA.....	12
2.2.3 <i>Polling</i>	12
2.3 Desenvolvimento dos códigos nos módulos Arduino para transmissão simultânea de dados.....	13
2.4 Desempenho do sistema com transmissão simultânea de dados.....	16
3 Autoavaliação	20
4 Conclusão	20
5 Referências	21

Índice de figuras e tabelas

Figura 1.1 - Exemplo de um master a enviar 'S' para o slave com resposta 'F' ^[1]	6
Figura 1.2 - Circuito usado na comunicação série entre cada Arduino e o respetivo TRF ^[2]	7
Figura 1.3 - Fluxograma do transmissor (teste 1).	8
Figura 1.4 - Fluxograma do recetor (teste 1).....	9
Figura 1.5 - Diagrama temporal do RTT (não está à escala).....	10
Figura 1.6 - Demonstração do RTT no programa ao serem enviados 32 caracteres.	10
Figura 1.7 - Demonstração de uma trama corrompida no programa.....	11
Figura 2.1 - Colisões de tramas enviadas por vários utilizadores com ALOHA.	12
Figura 2.2 - Fluxograma de cada transmissor (teste 2).	13
Figura 2.3 - Fluxograma do recetor (testes 2 e 3).	14
Figura 2.4 - Fluxograma de cada transmissor com o protocolo CSMA (teste 3)...	14
Figura 2.5 - Fluxograma de cada transmissor com o protocolo <i>pooling</i> (teste 4).	15
Figura 2.6 - Fluxograma do recetor com protocolo <i>polling</i> (teste 4).	16
Figura 2.7 - Resultado do recetor sem protocolo (teste 1).	17
Figura 2.8 - Resultado de ambos os transmissores com protocolo CSMA (teste 2).	18
Figura 2.9 - Resultado do recetor com protocolo CSMA (teste 2).	19
Figura 2.10 - Resultado de cada transmissor com protocolo <i>polling</i> (transmissor 1 à esquerda e transmissor 2 à direita) (teste 4).....	19
Figura 2.11 - Resultado do recetor com protocolo <i>polling</i> (teste 4).	20

1 Introdução

As tarefas desta segunda fase são de desenvolver comunicação por radiofrequência entre um módulo Arduino ligado a um TRF (*transceiver* de radiofrequência) do tipo rRF24L01+ e outro Arduino também ligado a um TRF do mesmo tipo com um protocolo de controlo de acesso ao meio (MAC – *Medium Access Control*).

Os objetivos da primeira tarefa incluem conhecer as características da interface de comunicação série entre cada TRF e o respetivo Arduino, efetuar as ligações físicas necessárias entre os componentes utilizados, conhecer e compreender o impacto no desempenho do sistema dos diferentes parâmetros configuráveis dos *transceivers* de RF, desenvolver o código nos módulos Arduino, para configuração e transmissão/receção de dados dos respetivos *transceivers* de RF e conceber e executar testes experimentais para avaliar o desempenho do sistema.

A segunda tarefa tem como objetivos a perceção das situações em que se torna necessária a utilização de um protocolo MAC, análise das vantagens e desvantagens dos diferentes tipos de protocolos de controlo de acesso ao meio para redes sem fios, implementação dos protocolos de controlo de acesso ao meio escolhidos pelo grupo sobre o código Arduino, e avaliação do desempenho do sistema com a transmissão simultânea de dados de dois dispositivos na mesma rede, comparando os resultados obtidos nos testes sem e com o protocolo MAC.

1 Tarefa 2.1 – Comunicação por radiofrequência Arduino-TRF-TRF-Arduino

1.1 Interface de comunicação série TRF-Arduino

A interface usada nesta fase para a comunicação série entre cada TRF e o respetivo Arduino é a SPI (*Serial Peripheral Interface*), que se trata de uma interface síncrona normalmente usada na transmissão de dados entre microcontroladores e pequenos periféricos como *shift registers*, sensores, e cartões SD (*Secure Digital Cards*). Esta interface usa linhas separadas de *clock* e de dados, juntamente com uma linha de seleção para escolher o dispositivo com que deseja comunicar ^[1].

Nesta interface, apenas um dos lados gera o sinal de *clock* (SCK – *Serial Clock*). Neste projeto, este lado tem o nome de “*master*”, enquanto o outro chama-se “*slave*”. Só pode haver um *master*, neste caso o Arduino, mas podem haver múltiplos *slaves*, neste caso só há um que é o TRF ^[1].

Quando os dados são enviados de um *master* para um *slave*, estes são enviados pela linha de dados MOSI (*Master Out / Slave In*). Se o *slave* precisar de enviar uma resposta de volta para o *master*, este continuará a gerar um número pré-definido de ciclos de *clock*, e o *slave* colocará os dados na linha de dados MISO (*Master In / Slave Out*) ^[1].

O número de ciclos de *clock* gerados pelo *master* é pré-definido porque o mesmo deve saber com antecedência quando é que o *slave* precisa de devolver dados e a quantidade desses dados, ao contrário da série assíncrona, onde quantidades aleatórias de dados podem ser enviadas em ambas direções em qualquer altura, o que na prática não é um problema, devido à SPI ser geralmente usada na comunicação com sensores que têm uma estrutura específica de comandos ^[1].

A SPI é “*full duplex*”, ou seja, tem separadas linhas de transmissão e receção, portanto, em certas situações, podem ser transmitidos e recebidos dados ao mesmo tempo ^[1].

Esta interface também dispõe de uma linha SS (*Slave Select*) que alerta o *slave* para este receber/enviar dados e é também usada quando estão presentes vários *slaves* para que seja selecionado aquele que deseja comunicar. As sequências de linhas SCK, MOSI, MISO e SS estão exemplificadas na Figura 1.1 ^[1].

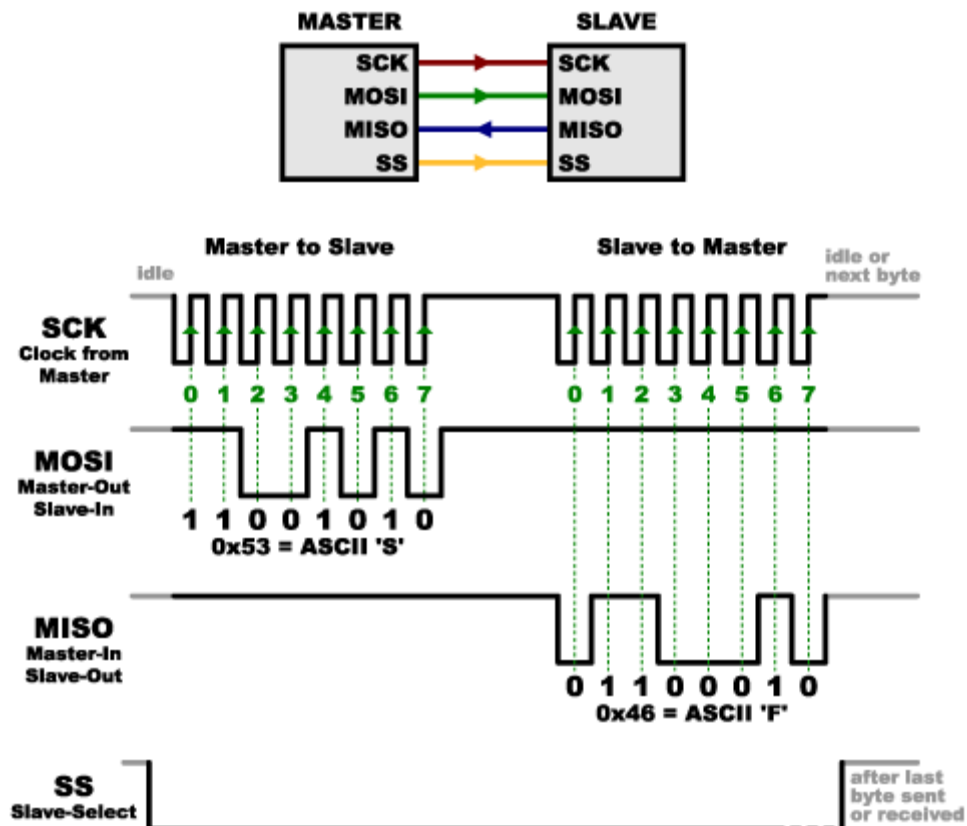


Figura 1.1 - Exemplo de um master a enviar 'S' para o slave com resposta 'F' ^[1].

A SPI tem a vantagem de ser mais rápida do que interfaces assíncronas, o *hardware* usado pode ser um simples *shift register* e suporta vários *slaves*. Contudo, é desvantajosa no sentido de precisar de mais linhas de sinais com fios comparado com outros métodos de comunicação, as comunicações devem ser bem definidas, ou seja, não podem ser enviadas quantidades aleatórias de dados a qualquer altura, o *master* deve controlar todas as comunicações, portanto os *slaves* não podem comunicar entre si diretamente, e normalmente são necessárias diferentes linhas SS para cada *slave*, o que pode ser um problema se forem necessários grandes números de *slaves* ^[1].

1.2 Ligações físicas

As ligações físicas necessárias para que haja comunicação entre múltiplos Arduinos são feitas através de um TRF ligado a cada um. Estas ligações são feitas através dos pinos 13 ↔ SCK, 12 ↔ MISO, ~11 ↔ MOSI, 8 ↔ CSN (*Chip Select Not*), 7 ↔ CE (*Chip Enable*), 3V3 (3.3 V) ↔ VCC (*Voltage Common Collector*) e GND (*Ground*) ↔ GND, como é demonstrado no esquema da Figura 1.2.

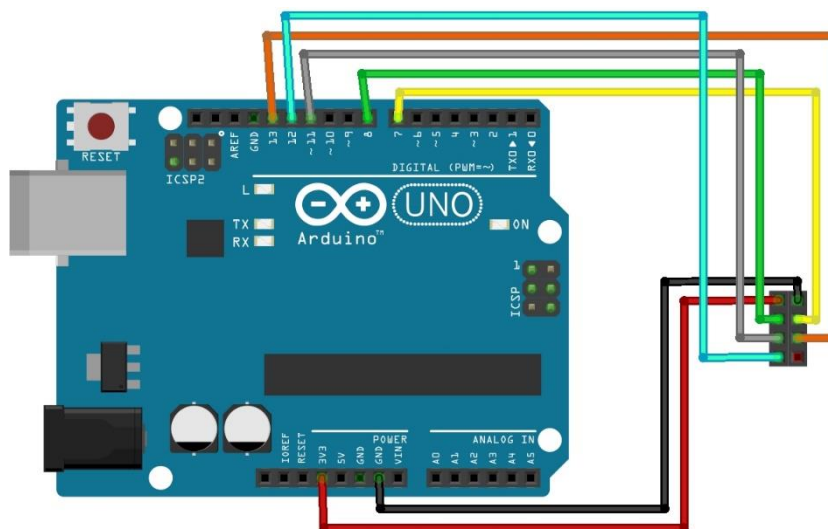


Figura 1.2 - Circuito usado na comunicação série entre cada Arduino e o respetivo TRF [2].

1.3 Impacto dos diferentes parâmetros configuráveis de TRF

O módulo nRF24L01+ dispõe de uma biblioteca configurável nos módulos Arduino com comandos providenciados pela sua API (*Application Programming Interface*) [3]. Através desses comandos, podem ser configurados diversos parâmetros para construção do programa e para controlo de operações durante a sua execução.

Os parâmetros principais para a construção do programa incluem a configuração dos pinos que estão ligados ao CE e CSN, a abertura dos *pipes* de escrita com um endereço definido para cada transmissor e de leitura para se receber dados do transmissor com o endereço definido, o número e o tempo de atraso de tentativas que devem ser executadas devido a uma transmissão falhada, o canal de comunicação RF que deve ser diferente do canal usado pelos outros grupos da UC (Unidade Curricular) para não haver interferência, o tamanho estático de cada *payload* (mensagem) enviado, o nível de amplificador de potência (PA – *Power Amplifier*), a taxa de transmissão, a presença de pacotes de conhecimento automático (se estiver desativado, podem haver colisões entre mensagens enviadas ao mesmo tempo para o mesmo recetor, e o tamanho e a validação do CRC (*Cyclic Redundancy Check*) que sendo um detetor de erros, deixa de prevenir os mesmos se estiver desativado.

Os parâmetros de controlo para operações durante a execução do programa incluem a leitura dos parâmetros definidos na configuração, a ativação e a paragem da escuta nos *pipes* de escrita abertos, a verificação de dados disponíveis para serem lidos, a escrita e transmissão de dados com um tamanho definido no *pipe* de leitura aberto, a leitura do *payload* recebido, a verificação da linha de transmissão para verificar se está ocupada por um *carrier* (que é o que acontece durante a transmissão de uma mensagem) que é útil para verificar se há interferências no canal em uso e a ativação de baixa e alta potência (em alta potência o TRF torna-se mais responsivo).

1.4 Desenvolvimento do código nos módulos Arduino

Foram desenvolvidos códigos nos módulos Arduino para transmissão de dados entre apenas um transmissor e um recetor representado nos fluxogramas da Figura 1.3 e da Figura 1.4.

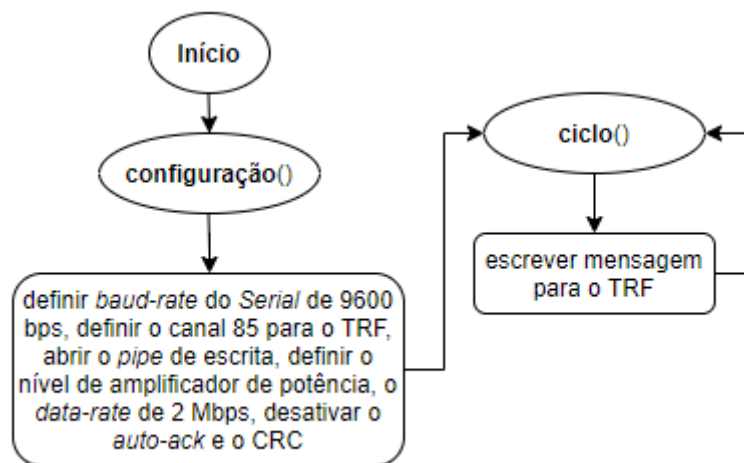


Figura 1.3 - Fluxograma do transmissor (teste 1).

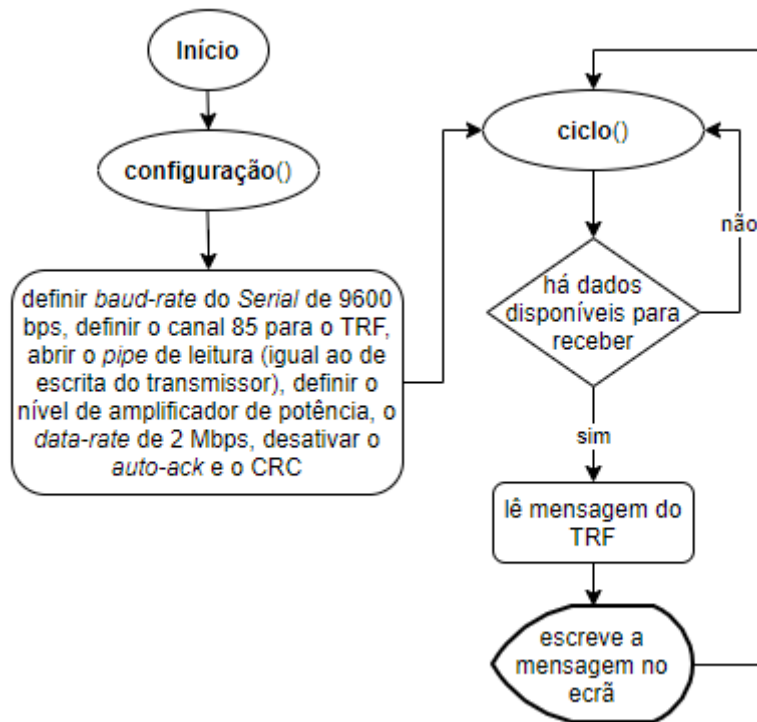


Figura 1.4 - Fluxograma do recetor (teste 1).

1.5 Testes experimentais

Os parâmetros relevantes que podem ser calculados teoricamente são o tempo de transmissão (T_{tx}) e o tempo de propagação (T_p).

Sabendo que o tamanho da trama (L) usado no programa é de 32 bytes, portanto $32 \times 8 = 256$ bits, e que a taxa de transmissão é de 2 Mbps, portanto $2 \times 10^6 = 2\,000\,000$ bps:

$$T_{tx} = \frac{L}{R} = \frac{256}{2\,000\,000} = 0,000128 \text{ s} = 128 \text{ } \mu\text{s}.$$

Além disso, os dados foram enviados a uma distância (d) de 1 m à velocidade da luz ($c = 299\,792\,458$ m/s), portanto:

$$T_p = \frac{d}{c} = \frac{1}{299\,792\,458} \approx 3,3356 \times 10^{-9} \text{ s} \approx 3,336 \text{ ns}.$$

Através dos valores calculados, pode-se também calcular o *round-trip time* (RTT) que é o tempo que os dados demoram a ser enviados e a obter uma resposta com o mesmo tamanho de volta, que pode ser calculado com a ajuda do diagrama temporal da Figura 1.5.

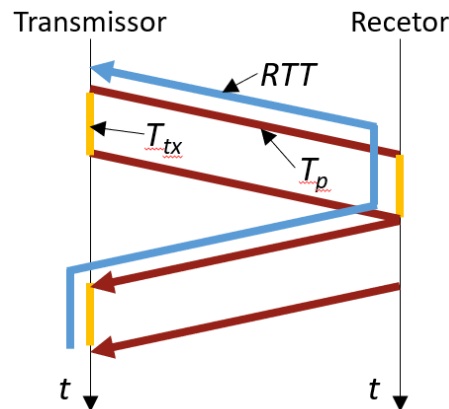


Figura 1.5 - Diagrama temporal do RTT (não está à escala).

Pode-se então calcular o RTT da seguinte maneira:

$$RTT = 2 \times T_{tx} + 2 \times T_p \approx 2 \times 0,000128 + 2 \times 3,3356 \times 10^{-9} \approx \\ \approx 2,56007 \times 10^{-4} \text{ s} = 256,007 \text{ } \mu\text{s}.$$

Experimentalmente, verificou-se na Figura 1.6 que o RTT variou entre de 1124 e 1132 μs ao enviar uma mensagem de 32 caracteres (os dois caracteres para identificar o pacote e o utilizador foram ocultados).

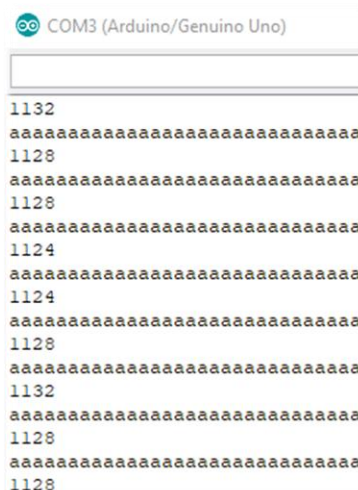


Figura 1.6 - Demonstração do RTT no programa ao serem enviados 32 caracteres.

Verificou-se também que ao serem enviadas 1000 mensagens consecutivamente, 3 chegaram corrompidas (no excerto do teste representado na Figura 1.7, pode-se ver que a 532ª trama contém caracteres indevidos), portanto a taxa de erro de pacote (PER – *packet error rate*) pode ser calculada da seguinte maneira:

$$PER = \frac{N_{\text{pacotes corrompidos/perdidos na camada}}}{N_{\text{pacotes transmitidos}}} = \frac{3}{1000} = 0,003.$$

```
COM10 (Arduino/Genuino Uno)
|
526
: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
527
: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
528
: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
529
: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
530
: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
531
: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
532
: 5aAaAaAaAaAaAaAaAaAaAaAaAaAaAa
533
: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
534
: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

Figura 1.7 - Demonstração de uma trama corrompida no programa.

Além disso, verificou-se que o número de erros aumentou consideravelmente a partir de uma distância de 20 m entre o transmissor e o recetor e que o mesmo deixou de receber dados a partir dos 70 m.

2 Tarefa 2.2 – Controlo de acesso ao meio

2.1 Situações em que se torna necessário utilizar um protocolo de acesso ao meio (MAC)

Numa rede de transmissão, o problema principal é de como determinar quem usa um canal quando há competição pelo mesmo. Por exemplo, consideremos uma chamada de conferência onde seis pessoas, cada um com o seu telefone, estão ligados a essa chamada para que todos consigam falar e ouvir entre si. É muito provável que quando um destes acaba de falar, dois ou mais vão começar a falar ao mesmo tempo, originando o caos. Numa reunião cara-a-cara, o caos é evitado por meios externos. Por exemplo, numa reunião, as pessoas levantam as suas mãos para pedirem permissão para falar. Quando há apenas um canal disponível, torna-se muito difícil determinar quem deve ser o próximo a usa-lo. Para resolver este problema, são necessários protocolos específicos, situados na subcamada MAC, que pertence à camada de ligação de dados. Estes protocolos servem para evitar colisões na transmissão de tramas ou detetar e recuperar das mesmas [4].

2.2 Protocolos MAC para redes sem fios

São conhecidos diversos algoritmos para alocar canais de acesso múltiplo, tendo estes as suas vantagens e desvantagens. Os protocolos que foram utilizados nesta fase do projeto foram ALOHA, CSMA (*Carrier Sense Multiple Access*) e *polling*.

2.2.1 ALOHA

O sistema ALOHA é um protocolo de acesso aleatório cujo objetivo é de deixar os utilizadores transmitirem sempre que houver dados para enviar. O problema é que isto

causa colisões, fazendo com que as tramas colididas fiquem danificadas. Quando cada transmissor envia a sua trama para o recetor, este envia uma resposta de volta para os transmissores verificarem se as suas tramas foram enviadas devidamente. Exemplos de colisões estão representados na Figura 2.1 ^[4].

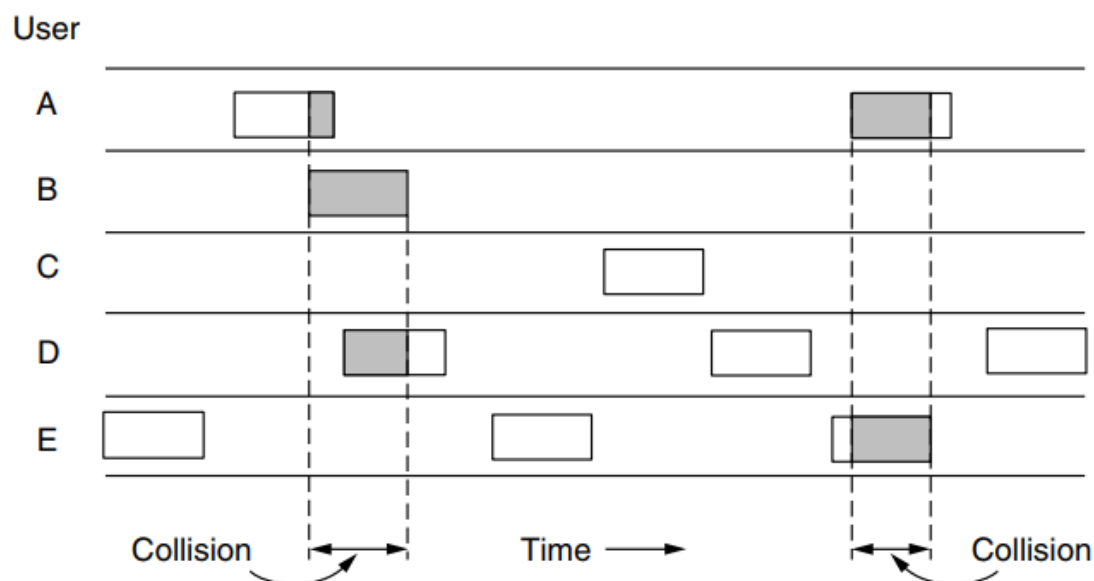


Figura 2.1 - Colisões de tramas enviadas por vários utilizadores com ALOHA.

Concluindo, as vantagens do ALOHA são de transmissão imediata e as suas desvantagens são de risco elevado de colisão de dados enviados por múltiplos transmissores ao mesmo tempo.

2.2.2 CSMA

Assim como o sistema ALOHA, o CSMA trata-se também de um protocolo de acesso aleatório, mas ao contrário do anterior, quando um transmissor tem dados para enviar, antes de o fazer, este verifica se o canal está ocupado por outro transmissor que esteja a enviar no momento. Se estiver livre, os dados são enviados, senão, o transmissor espera um tempo aleatório e verifica de novo, e o processo é repetido até que o canal esteja livre. As colisões são muito raras, contudo podem acontecer no caso de várias tramas serem enviadas ao mesmo tempo, pois ambos os transmissores verificam que o canal está livre ^[4].

Portanto, as vantagens do CSMA são de transmissão simultânea com raros casos de colisões não detetadas e as suas desvantagens são de desempenho reduzido quando são detetadas colisões.

2.2.3 Polling

Ao contrário dos protocolos anteriores, o *polling* é um protocolo de agendamento em que o recetor dá a ordem a um transmissor de cada vez para enviar. Portanto, cada transmissor tem um identificador e o recetor envia para todos os transmissores um identificador, o transmissor com esse identificador é quem tem permissão para enviar a sua

mensagem. Quando o recetor recebe essa mensagem, este dá permissão ao transmissor seguinte para transmitir, e assim sucessivamente para os restantes transmissores até voltar o primeiro transmissor, reiniciando o ciclo ^[5].

As vantagens do *polling* são de ausência de colisões, contudo tem as desvantagens de causar sobrecarga no sistema e de ter um único ponto de falha que é o recetor, ou seja, se este falhar, o sistema falha por completo.

2.3 Desenvolvimento dos códigos nos módulos Arduino para transmissão simultânea de dados

Foram desenvolvidos códigos nos módulos Arduino para transmissão de dados entre dois transmissores e um recetor sem nenhum protocolo aplicado, representado nos fluxogramas da Figura 2.2 e da Figura 2.3.

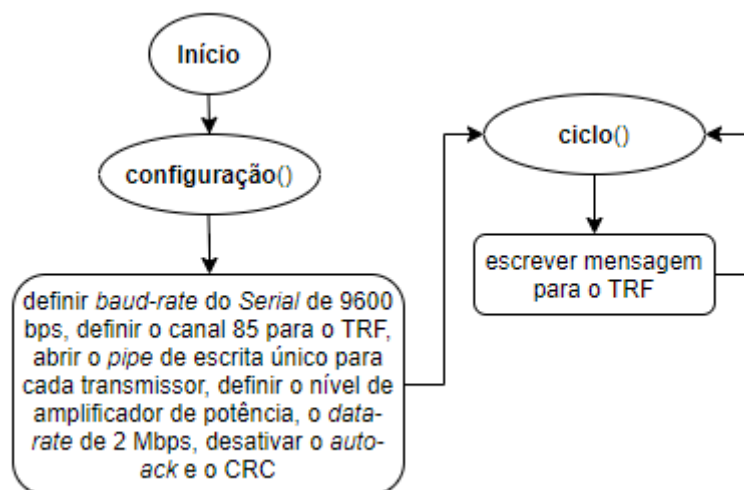


Figura 2.2 - Fluxograma de cada transmissor (teste 2).

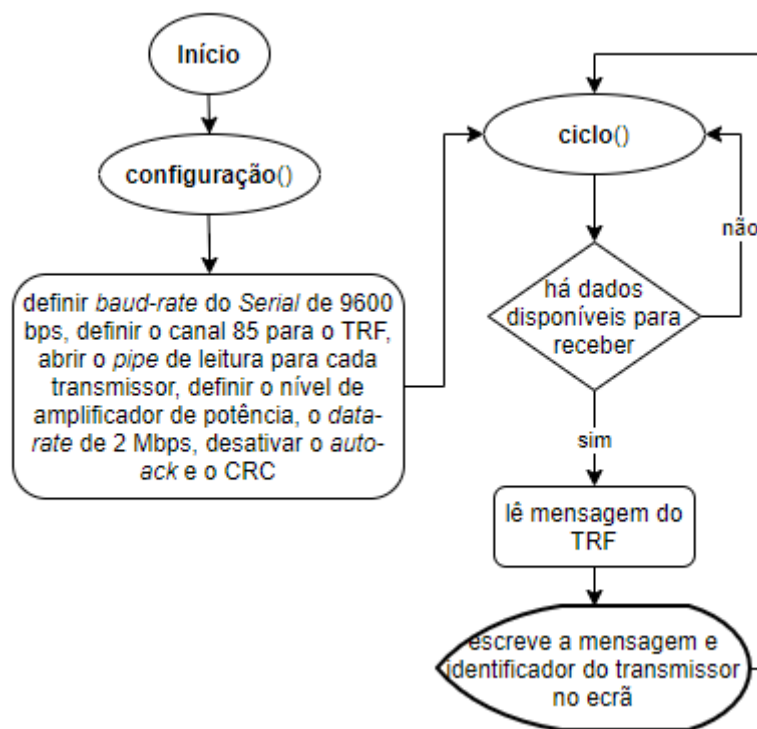


Figura 2.3 - Fluxograma do recetor (testes 2 e 3).

Aplicou-se no código de cada transmissor o protocolo CSMA para detetar e evitar colisões entre as mensagens de cada um, representado no fluxograma da Figura 2.4.

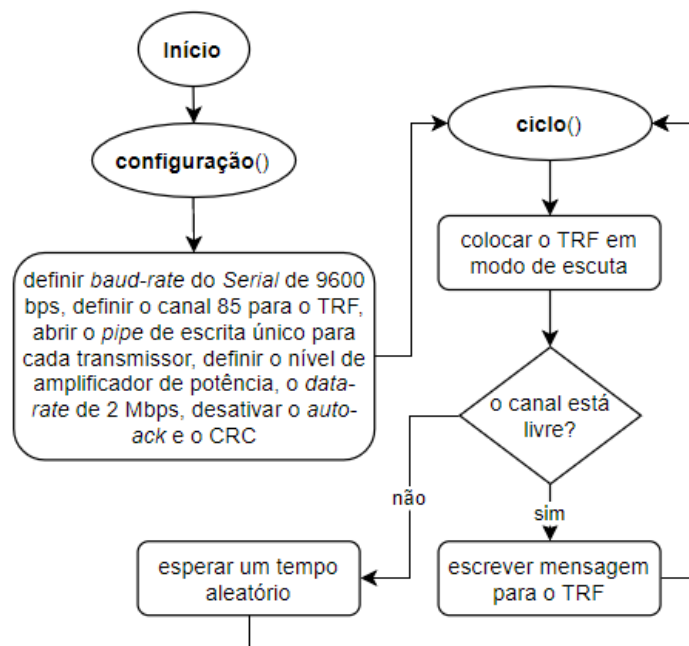


Figura 2.4 - Fluxograma de cada transmissor com o protocolo CSMA (teste 3).

Finalmente, aplicou-se no código de cada transmissor e recetor o protocolo *polling* para dar a vez a cada transmissor para enviar a sua mensagem, evitando colisões. O fluxograma de cada transmissor está representado na Figura 2.5 e o do recetor na Figura 2.6.

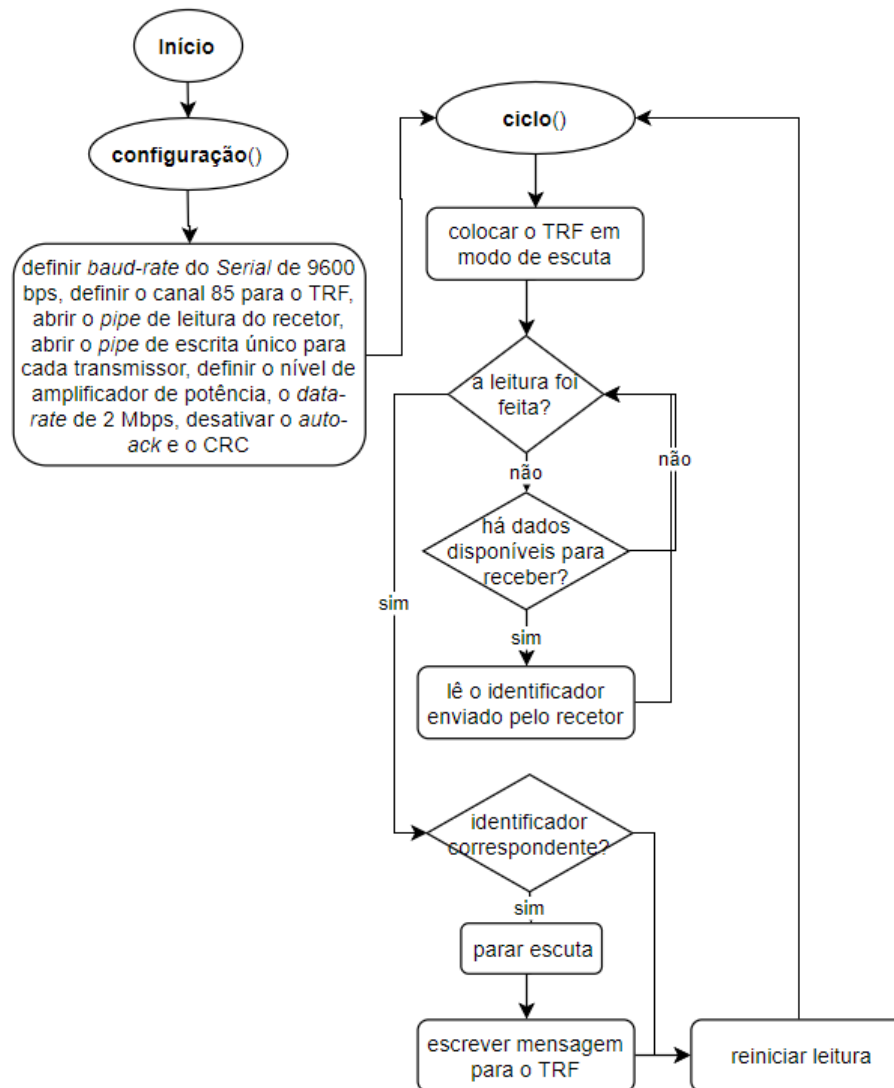


Figura 2.5 - Fluxograma de cada transmissor com o protocolo *pooling* (teste 4).

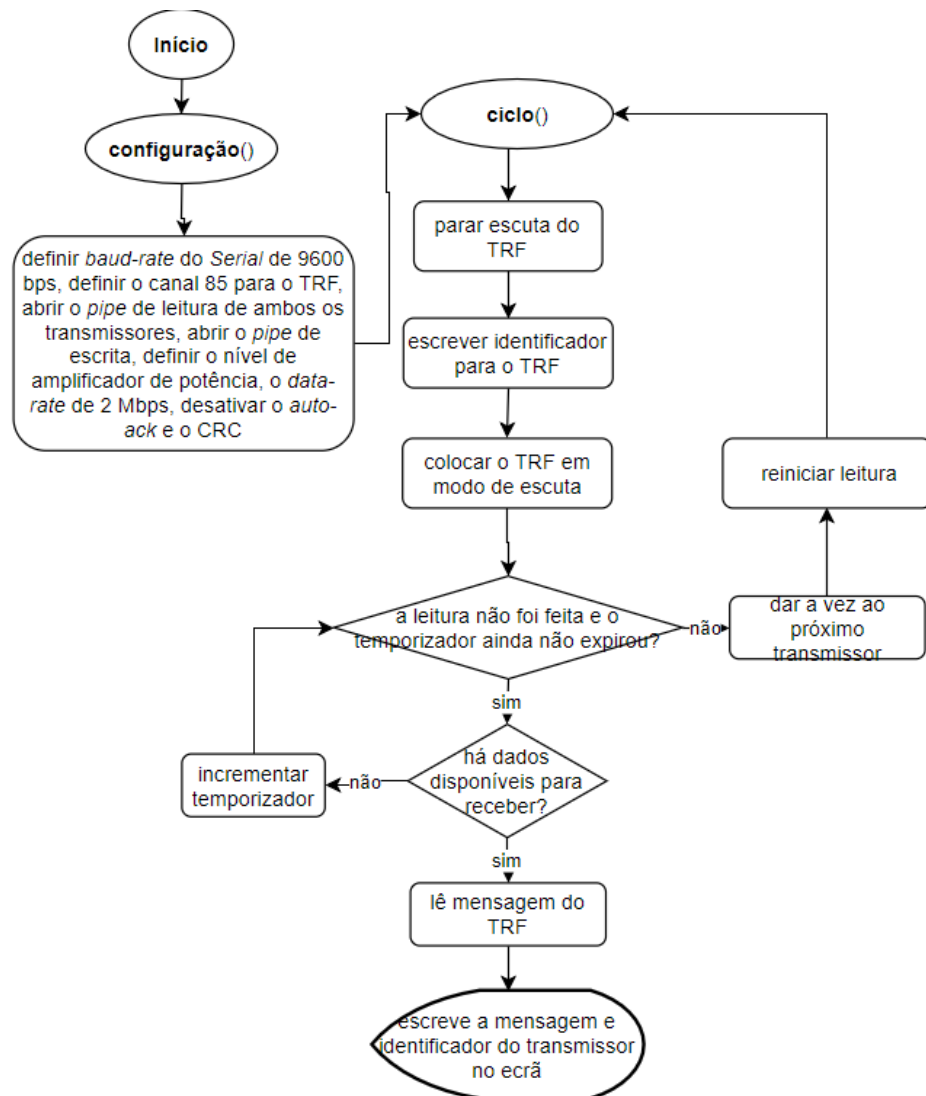


Figura 2.6 - Fluxograma do recetor com protocolo *polling* (teste 4).

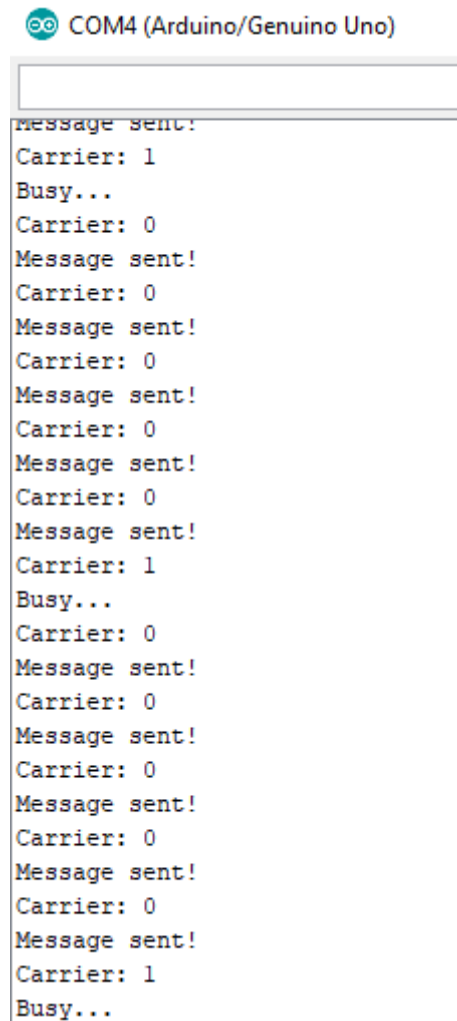
2.4 Desempenho do sistema com transmissão simultânea de dados

Testou-se o desempenho do recetor ao receber as mensagens de ambos os transmissores sem nenhum protocolo, obtendo-se os resultados da Figura 2.7.

```
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaa"???"
User 1: aaaaaaa`_???U*???W???¿□□
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

Figura 2.7 - Resultado do recetor sem protocolo (teste 1).

Ao ser aplicado o protocolo CSMA no código de cada transmissor, foram obtidos os resultados da Figura 2.8 e da Figura 2.9.



COM4 (Arduino/Genuino Uno)

```
Message sent!  
Carrier: 1  
Busy...  
Carrier: 0  
Message sent!  
Carrier: 0  
Message sent!  
Carrier: 0  
Message sent!  
Carrier: 0  
Message sent!  
Carrier: 0  
Message sent!  
Carrier: 1  
Busy...  
Carrier: 0  
Message sent!  
Carrier: 0  
Message sent!  
Carrier: 0  
Message sent!  
Carrier: 0  
Message sent!  
Carrier: 0  
Message sent!  
Carrier: 1  
Busy...
```

Figura 2.8 - Resultado de ambos os transmissores com protocolo CSMA (teste 2).

```
COM10 (Arduino/Genuino Uno)

User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

Figura 2.9 - Resultado do recetor com protocolo CSMA (teste 2).

Ao ser aplicado o protocolo *polling* em ambos os transmissores e no recetor, foram obtidos os resultados da Figura 2.10 e da Figura 2.11 (cada transmissor verifica se o número recebido do recetor (User 0) é o que lhe dá permissão para enviar a sua mensagem).

<pre>COM3 (Arduino/Genuino Uno) User 0: 2 User 0: 1 Message Sent! User 0: 2 User 0: 1 Message Sent! User 0: 2 User 0: 1 Message Sent! User 0: 2 User 0: 1 Message Sent!</pre>	<pre>COM4 (Arduino/Genuino Uno) User 0: 1 User 0: 2 Message sent! User 0: 1 User 0: 2 Message sent! User 0: 1 User 0: 2 Message sent! User 0: 1 User 0: 2 Message sent!</pre>
--	--

Figura 2.10 - Resultado de cada transmissor com protocolo *polling* (transmissor 1 à esquerda e transmissor 2 à direita) (teste 4).

```
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
User 1: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
User 2: bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

Figura 2.11 - Resultado do recetor com protocolo *polling* (teste 4).

3 Autoavaliação

O André Almeida desenvolveu grande parte dos códigos do recetor de cada teste, assim como alguma parte dos transmissores, fez parte dos fluxogramas dos códigos que desenvolveu, implementou as ligações físicas necessárias, estudou os parâmetros da API do TRF e os protocolos MAC e fez grande partes dos testes experimentais

O Leandro Alves desenvolveu grande parte dos códigos dos transmissores de cada teste, assim com alguma parte do recetor, ajudou a implementar as ligações físicas necessárias, pesquisou parte das características do SPI e dos protocolos MAC, também estudou os parâmetros da API do TRF e fez parte dos testes experimentais.

O Eduardo Santos ajudou a desenvolver parte dos códigos dos transmissores de cada teste, pesquisou grande parte das características do SPI e dos protocolos MAC, estudou os parâmetros da API do TRF, fez os testes teóricos e ajudou a fazer os experimentais e fez grande parte dos fluxogramas dos códigos desenvolvidos.

4 Conclusão

Esta fase ajudou-nos a atingir os objetivos propostos que incluem o conhecimento da interface SPI, das características do TRF. Conseguimos também estabelecer a comunicação entre Arduinos utilizando o TRF, enviando dados simultaneamente com dois

transmissores e um recetor usando protocolos MAC para corrigir colisões entre as mensagens desses transmissores.

Neste trabalho tivemos algumas dificuldades, incluindo o desconhecimento de que o módulo TRF tinha o protocolo CRC ativo por definição, até que aprendemos sobre o mesmo numa aula da UC de Redes de Computadores I, a implementação do protocolo CSMA, pois não estavam a ser detetadas colisões entre as mensagens e, por último, o teste experimental do RTT. Apesar destes obstáculos, acreditamos que concluímos esta fase com sucesso.

5 Referências

- [1] Serial Peripheral Interface (SPI) – learn.sparkfun.com.
<https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all>
05/11/2018.
- [2] [SOLVED] nRF24L01+ overheating?.
<http://forum.arduino.cc/index.php?topic=542499.0>
11/10/2018.
- [3] RF24: RF24 Class Reference.
<https://maniacbug.github.io/RF24/classRF24.html>
11/10/2018.
- [4] A. S. Tanenbaum, “Computer Networks”, Prentice Hall, 2003.
- [5] Multi-drop polling basics from Pulse Supply.
http://www.pulsesupply.com/data101/multidrop_polling_basics.htm
10/11/2018.